

CO3: AT₃

**SIMATS ENGINEERING
ASSESSMENT TOOLS**

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO3: Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL : 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5
Duration: 1 Hour

Total Marks: 25

CASE STUDY

Code of Conduct:

I V. Smylika (192372115) (Name / Reg No) certify that this submission is my original work and I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Smylika ✓
Signature of the student:

1. Consider a case study in parsing nested structured expressions using a Context-Free Grammar.
Given the grammar

$S \rightarrow (L) \mid a$

$L \rightarrow L, S \mid S,$

Ambiguous Grammar

The initial grammar is:

$$E \rightarrow E + E \mid E * E \mid (E) \mid id$$

The grammar is ambiguous because an expression such as $a + b * c$ can have more than one parse tree. It does not clearly specify whether $+$ or $*$ should be evaluated first.

2. Optimized Context-Free Grammar

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow (E) \mid id$$

3. Precedence Handling

- E handles addition and subtraction.
- T handles multiplication and division.
- F handles parentheses and identifiers.
- Therefore, multiplication/div have higher precedence than add/sub.
- Parentheses have the highest priority.

For example: $a + b * c$ is represented as: $a + (b * c)$ rather than $(a + b) * c$

4. Optimization performed

1. Separate non-terminals are used for different precedence levels.
2. T handles $*$ and $/$.
3. E handles $+$ and $-$.
- F handles parentheses and identifiers.
5. The ambiguity in the original grammar is removed.
6. The grammar is suitable for constructing efficient parser.

2. Mini programming Language parser Development

Case study:

A startup is designing a small scripting language for automation. The parser frequently reports syntax conflicts during compilation.

Initial Grammar with Left Recursion

$stmt \rightarrow stmt\ stmt \mid stmt$

This grammar contains left recursion because $stmt$ appears as symbol on the right-hand side. This can cause problems for predictive parsers.

Optimized CFG

The left recursion can be removed as follows:

$stmt \rightarrow stmt\ stmt'$

$stmt' \rightarrow stmt\ stmt' \mid$

Here, $stmt'$ allows zero or more additional statements.

CFG for Language Constructs

Loop:

$Loop \rightarrow while (condition) \{ stmt \}$

cond stat:

$Cond \rightarrow if (condition)$

Grammar Optimization for SQL Query Validation

Case study:

A database company is developing a query validation tool for commands.

Initial Grammar:

$Q \rightarrow SELECT\ A\ FROM\ T\ WHERE\ C$

Optimized CFG

$Q \rightarrow SELECT\ fields\ FROM\ table\ OPT\ where \rightarrow$

IoT devices

here, Optwhere makes the WHERE clause optimal.
Optimization performed

- Introduction Optimal productions using ϵ .
- Reduced grammar redundancy.
- Simplified parser design.
- Allows queries both with and without a WHERE clause.

4. Natural Language processing chatbot

Case Study:

A chatbot processes simple English sentences such as "students learn

CFG design

$S \rightarrow NPVP$

$VP \rightarrow Verb NP$

$Verb \rightarrow learn$

$NP \rightarrow Noun$

$Noun \rightarrow Students TOC$

Optimization performed

- Restricted the sentence structure to subject $\rightarrow$ verb $\rightarrow$ object
- Rejects invalid word ~~arrangements~~.
- Improves grammatical correctness.
- Makes sentence validation simpler.

Valid Sentence

Students learn TOC

Invalid Sentence

Learn students TOC

The invalid sentence is rejected because it does not
grammar:

Subject + Verb + Object

5- Parser Design for HTML-like Markup Language

Case study:

A web editor company is building a parser for a simplified language with nested tags.

CFG for Nested Tags

$S \rightarrow \langle \text{tag} \rangle S \langle / \text{tag} \rangle \mid \text{text} \mid \epsilon$

This grammar allows:

- opening and closing tags.
- Nested tags.
- Text content.
- Empty content.

Ex: $\langle \text{tag} \rangle$
 $\langle \text{tag} \rangle \text{text} \langle / \text{tag} \rangle$
 $\langle / \text{tag} \rangle$

Optimization performed

- Supports recursive nesting of tags.
- Ensures every opening tag has a corresponding closing tag.
- Allows text inside tags.
- Allows empty tags through ϵ .
- Makes validation of nested structures easier.

Valid Ex:

$\langle \text{tag} \rangle \text{text} \langle / \text{tag} \rangle$

Nested Valid

$\langle \text{tag} \rangle$

$\langle \text{tag} \rangle$

$\langle / \text{tag} \rangle$

Invalid Example

$\langle \text{tag} \rangle$

$\langle \text{tag} \rangle \text{text} \langle / \text{tag} \rangle$

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

92 / 100

J